UNLOX MINOR PROJECT 7 REPORT

CNN Image Classification and Streamlit Application

Project Title: Convolutional Neural Network (CNN) Digit Classification & Interactive Streamlit Academic Calculator

Name: Vansh Modi

Batch: Artificial Intelligence Program - September 2026 Batch - A

Selected Dataset (Part A): MNIST Handwritten Digits Dataset (Classes 0 to 9)

Streamlit App Topic (Part B): Student Academic Marks & Grade Calculator

OBJECTIVE & CNN DATASET/PREPROCESSING

2. **Objective:**

- **Part A Objective (CNN Image Classification):** The primary objective of Part A is to design, train, and evaluate a deep learning Convolutional Neural Network (CNN) capable of classifying grayscale images of handwritten digits (from 0 to 9) using the MNIST dataset. This solves the fundamental computer vision problem of automated optical digit recognition across diverse human handwriting styles.

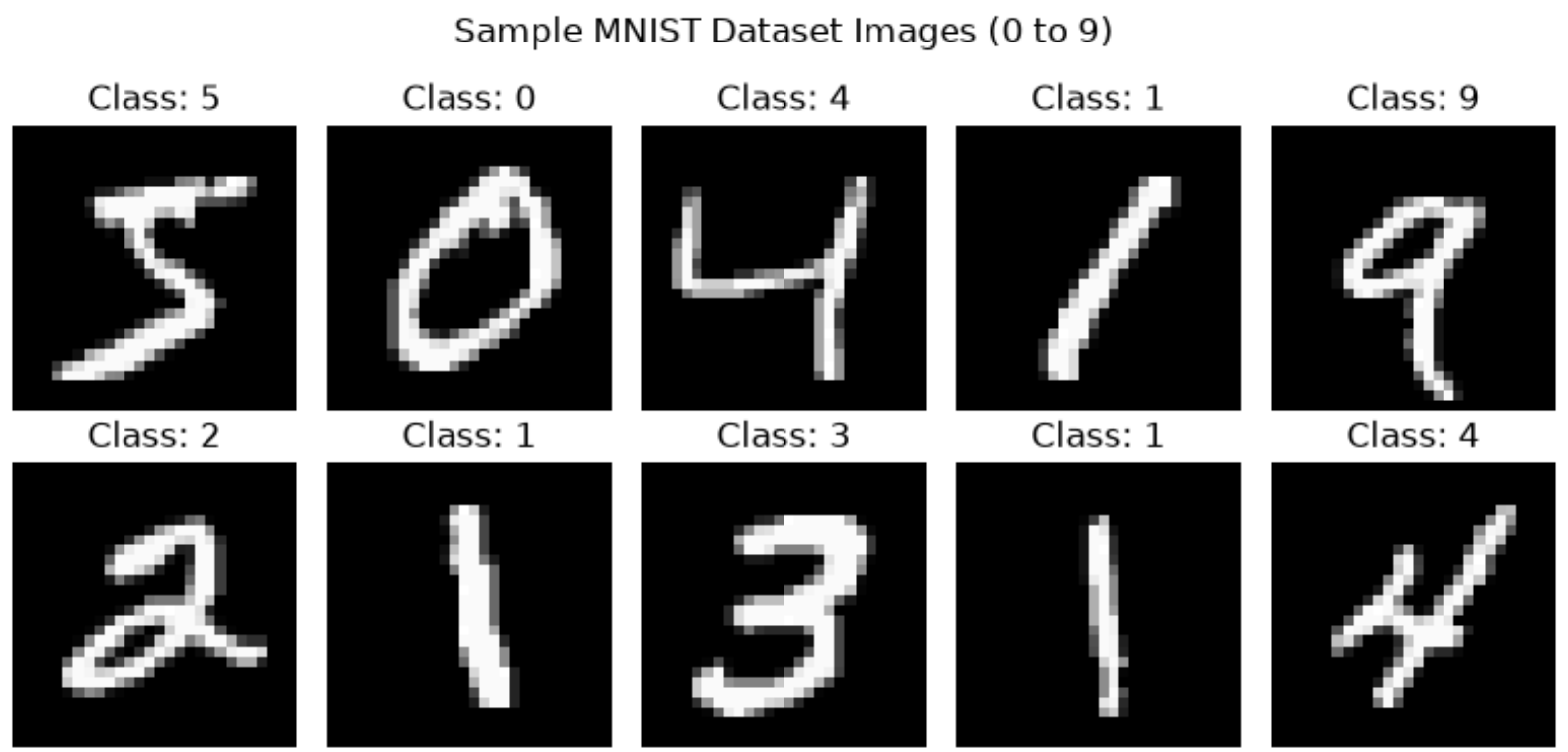
- **Part B Objective (Streamlit Application):** The objective of Part B is to build and deploy a clean, interactive Streamlit web application—specifically a **Student Academic Marks & Grade Calculator**. It solves the problem of manual grade computation by accepting student marks across five core subjects, validating user inputs against errors, and automatically computing total score, percentage, letter grade, and pass/fail status.

3. **CNN Workflow (Part A):**

Dataset Description and Class Names:

The MNIST (Modified National Institute of Standards and Technology) dataset is a benchmark computer vision dataset consisting of 70,000 grayscale images of handwritten digits:

- **Training Set:** 60,000 images (split into 54,000 for training and 6,000 for validation).
- **Testing Set:** 10,000 unseen images used strictly for final model evaluation.
- **Image Shape:** (1, 28, 28) — 1 grayscale color channel with a spatial resolution of 28×28 pixels per image.
- **Class Names (10 Classes):** '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'.



Data Preprocessing:

1. **Pixel Normalization:** Raw pixel intensities ranging from 0 to 255 were converted into floating-point tensors normalized to the range [0.0, 1.0]. Normalization ensures uniform feature scale and faster, stable convergence during gradient descent training.
2. **Train-Validation Split & Batching:** The 60,000 training images were split using a 90:10 ratio into 54,000 training samples and 6,000 validation samples, loaded in mini-batches of size 64 to balance training speed and gradient stability.

CNN ARCHITECTURE, TRAINING & CURVES

CNN Model Architecture and Training Approach:

The custom CNN architecture (MNIST_CNN) was constructed with two convolutional feature-extraction blocks followed by a fully connected classification head:

- **Convolutional Block 1:** A Conv2d layer with 32 filters (3×3 kernel) and ReLU activation to detect low-level edges and strokes, followed by MaxPool2d (2×2 pool size, stride 2) to reduce spatial dimensions from 28×28 to 13×13.
- **Convolutional Block 2:** A Conv2d layer with 64 filters (3×3 kernel) and ReLU activation to capture complex curves and loops, followed by MaxPool2d (2×2) to downsample feature maps to 5×5.
- **Classification Head:** A Flatten layer converts the (64, 5, 5) feature maps into a 1D vector of 1,600 features, connected to a hidden Linear layer with 128 neurons (ReLU), a Dropout(0.3) layer (30% dropout rate to prevent overfitting), and a final Linear output layer

with 10 neurons corresponding to the 10 digit classes.

- **Compilation & Training:** Trained for **5 epochs** using CrossEntropyLoss and the Adam optimizer (learning_rate = 0.001).

```
(.venv) apple@Vanshs-Apples-MacBook-Pro UNLOX Minor Project 7 % python Vansh_CNN_Streamlit_Project.py

 Step 1: Dataset Information
Training Sample Count : 60000
Testing Sample Count  : 10000
Single Image Shape    : (1, 28, 28) [1 Grayscale Channel, 28x28 pixels]
Number of Classes     : 10 -> ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
-> Saved '1_sample_images.png' for PDF Report.

 Step 2: Preprocessing & Train/Validation Split
Pixel values normalized from [0, 255] to [0.0, 1.0].
Train samples: 54000 | Validation samples: 6000 | Test samples: 10000

 Step 3: CNN Model Architecture & Summary
MNIST_CNN(
  (conv_layers): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc_layers): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1600, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=128, out_features=10, bias=True)
  )
)
Total Trainable Parameters: 225,034

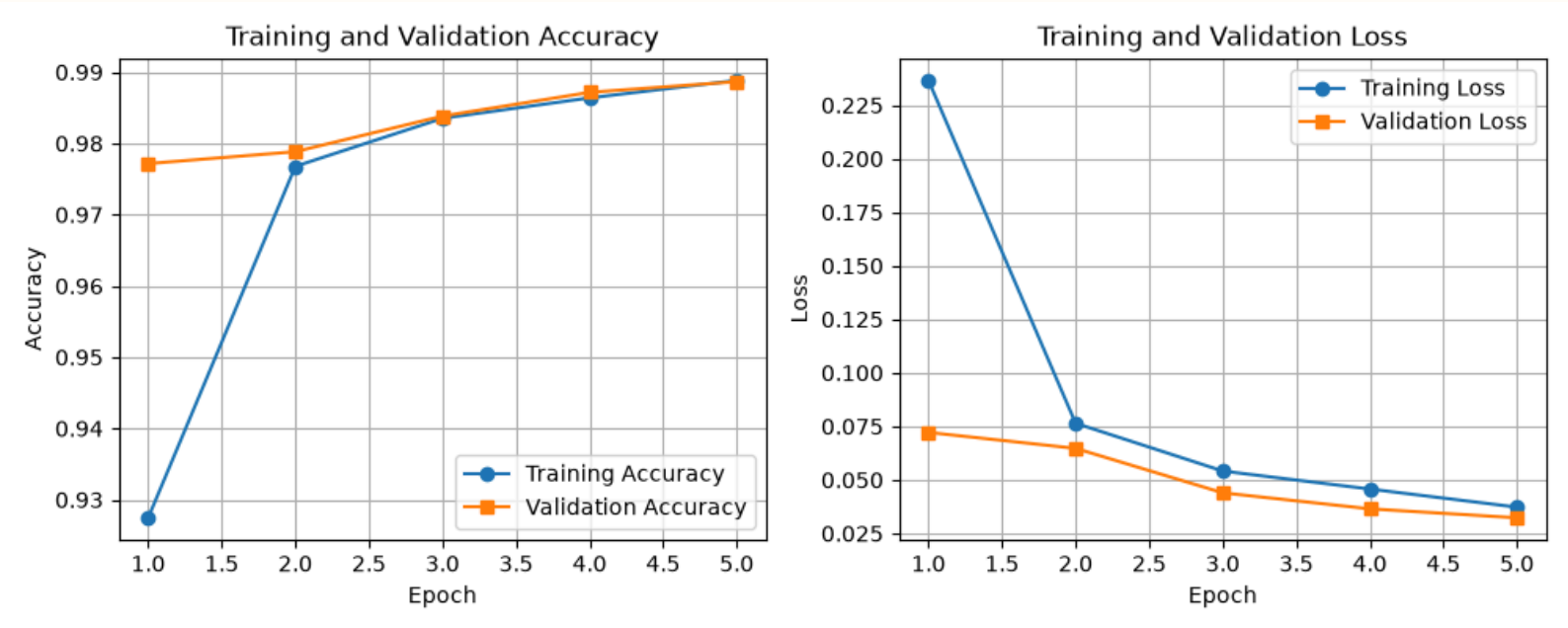

 Training CNN Model (5 Epochs)
Epoch [1/5] -> Train Loss: 0.2365, Train Acc: 92.75% | Val Loss: 0.0722, Val Acc: 97.72%
Epoch [2/5] -> Train Loss: 0.0764, Train Acc: 97.68% | Val Loss: 0.0647, Val Acc: 97.88%
Epoch [3/5] -> Train Loss: 0.0541, Train Acc: 98.35% | Val Loss: 0.0439, Val Acc: 98.38%
Epoch [4/5] -> Train Loss: 0.0457, Train Acc: 98.64% | Val Loss: 0.0365, Val Acc: 98.72%
Epoch [5/5] -> Train Loss: 0.0373, Train Acc: 98.88% | Val Loss: 0.0323, Val Acc: 98.87%
-> Saved '2_training_curves.png' for PDF Report.

 Step 4: Evaluating Model on Unseen Test Data
Final Test Accuracy : 99.16%
Final Test Loss     : 0.0261

 Step 5 & 6: Generating Sample Predictions & Explanation
-> Saved '3_predictions.png' for PDF Report.
Explanation: The CNN achieves ~99% accuracy on test images. Rare errors occur when handwritten digits are ambiguous or incomplete.
```

Training and Validation Accuracy & Loss:

Across the 5 training epochs, training and validation accuracy rapidly climbed above 98%, while training and validation loss decreased smoothly without diverging, confirming that the CNN learned generalizable features without overfitting.



STREAMLIT APPLICATION

4. Streamlit Application (Part B):

App Purpose and Overview:

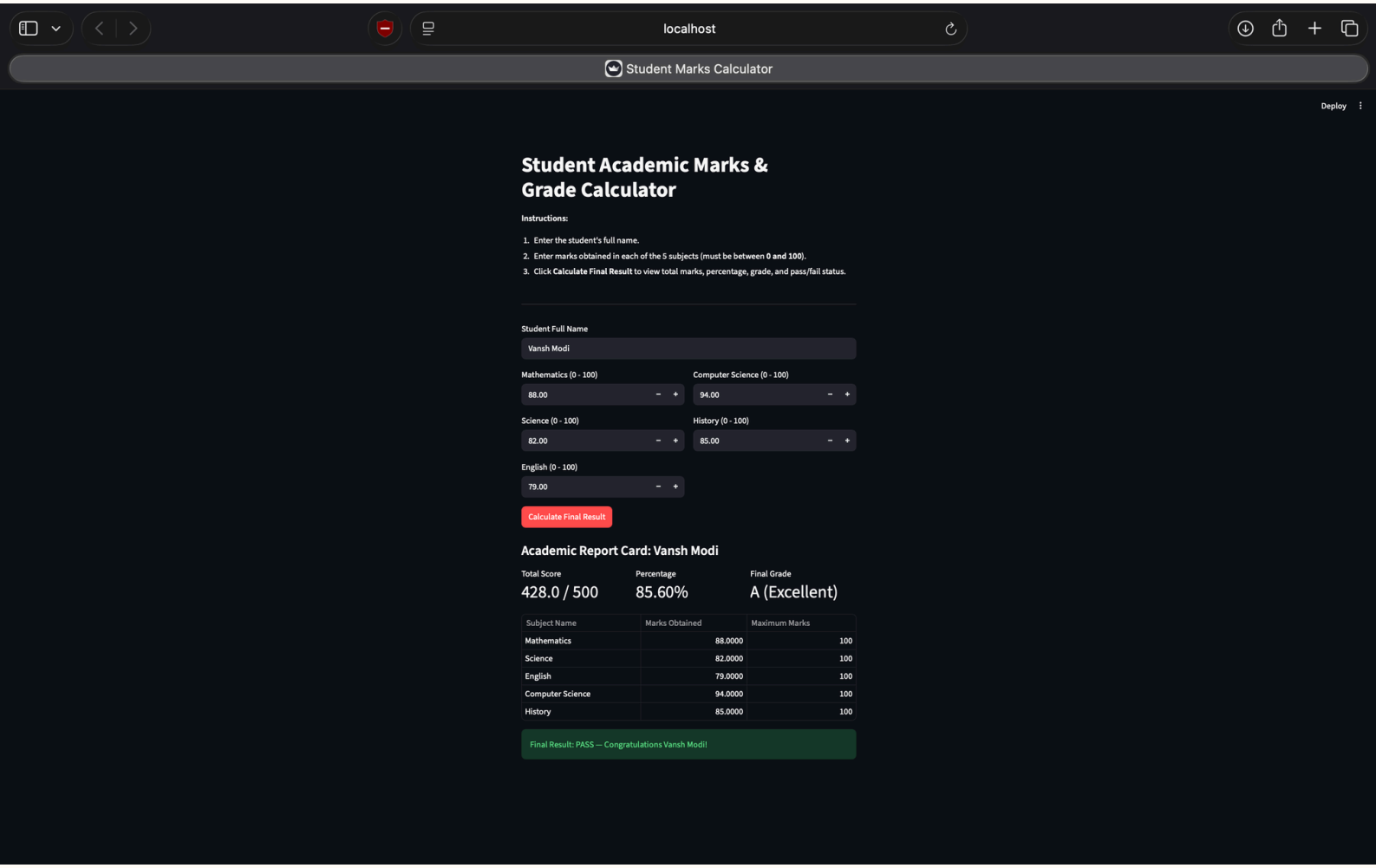
The **Student Academic Marks & Grade Calculator** is an interactive web tool built with Streamlit that enables students and educators to compute academic performance metrics instantaneously through a clean browser interface.

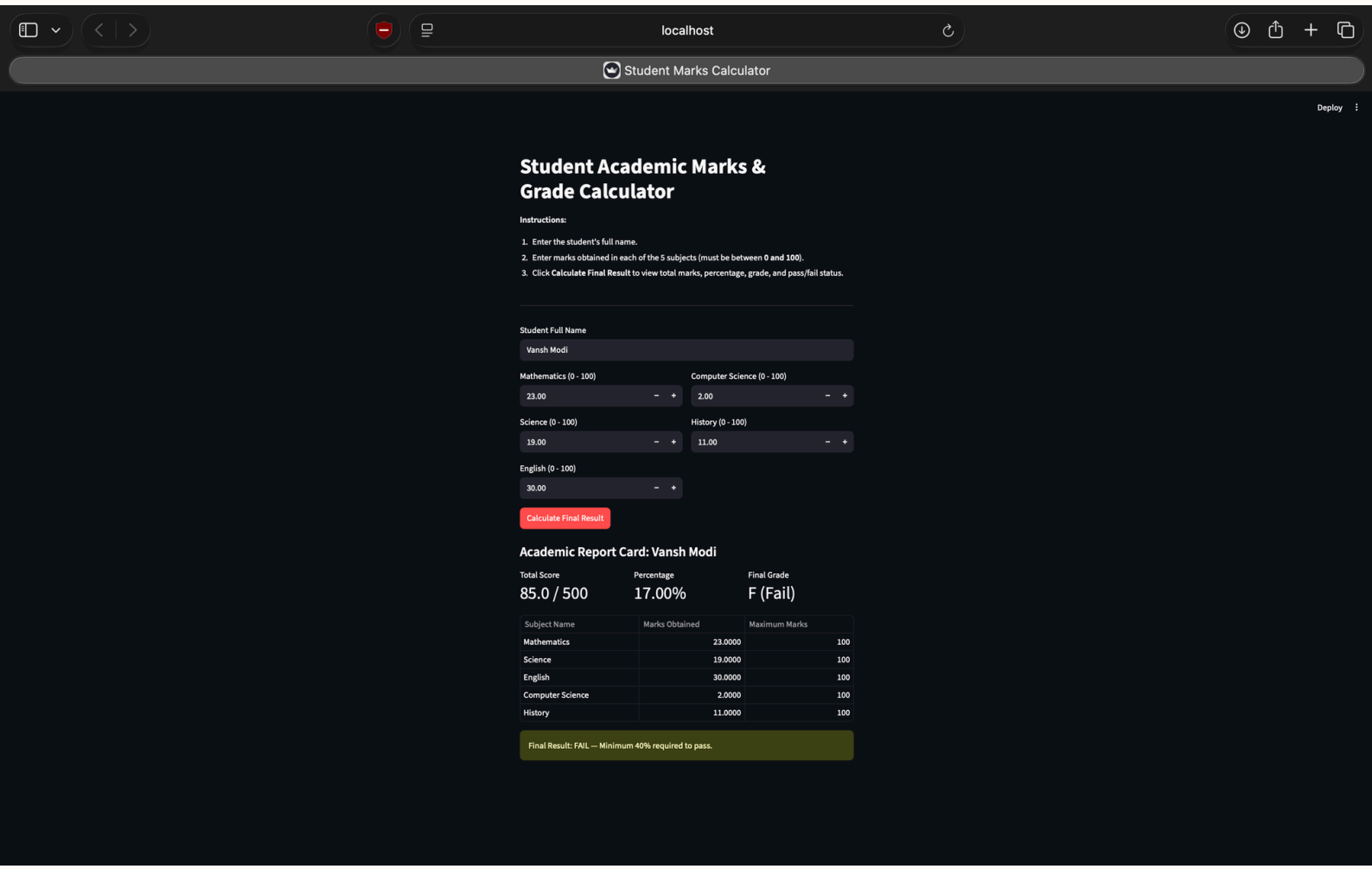
User Inputs, Processing, and Features:

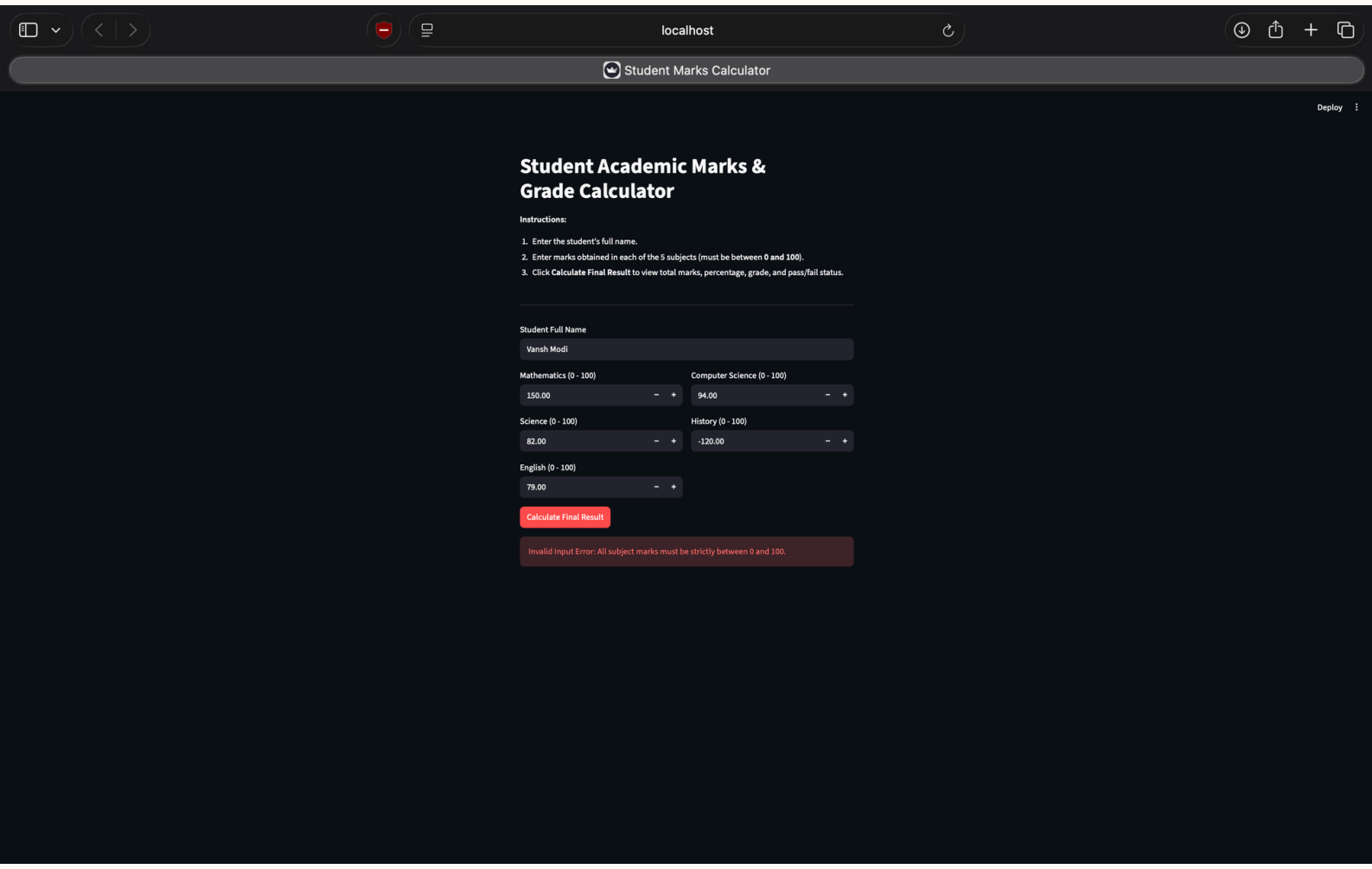
- **Clear Title & Instructions:** Displays step-by-step instructions guiding the user on valid input ranges (0 to 100).
- **User Inputs:** Includes a text input field for the **Student Full Name** and five structured numeric inputs (arranged in a two-column layout) for **Mathematics, Science, English, Computer Science, and History**.
- **Useful Calculation & Grading Logic:** Upon clicking **Calculate Final Result**, the app sums the five subject scores out of 500, computes the overall percentage, and assigns an academic grade based on standard thresholds (A+ for ≥90%, A for ≥80%, B for ≥70%, C for ≥60%, D for ≥40%, and F for

<40%).

- **Clean Output Presentation:** Displays three highlight metric cards (**Total Score**, **Percentage**, **Final Grade**), a formatted tabular breakdown of subject marks, and a color-coded status banner (PASS in green or FAIL in yellow).
- **Basic Invalid-Input Handling:** If the student name field is left blank or any subject mark is entered below 0 or above 100, the app blocks calculation and renders a descriptive red error banner (st.error).



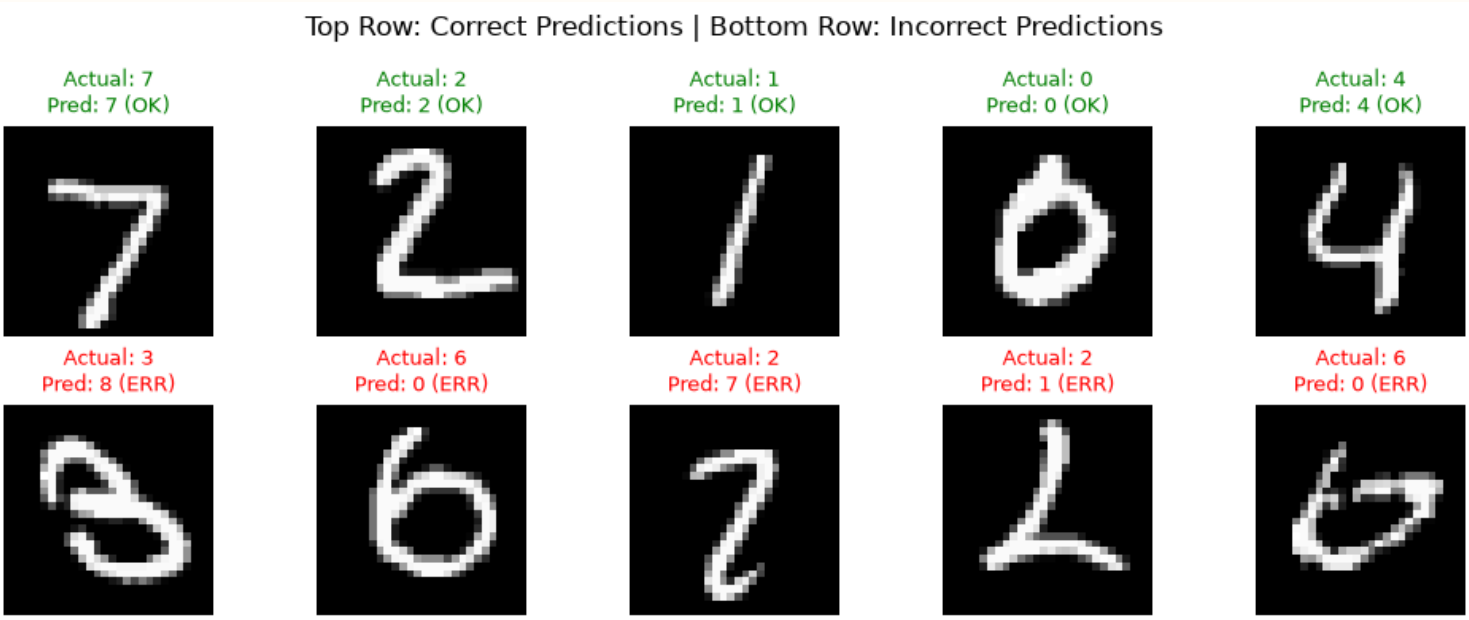

localhost
Student Marks Calculator
Deploy
Student Academic Marks & Grade Calculator
Instructions:
1. Enter the student's full name.
2. Enter marks obtained in each of the 5 subjects (must be between 0 and 100).
3. Click Calculate Final Result to view total marks, percentage, grade, and pass/fail status.
Student Full Name
Vansh Modi
Mathematics (0 - 100)
23.00
Computer Science (0 - 100)
2.00
Science (0 - 100)
19.00
History (0 - 100)
11.00
English (0 - 100)
30.00
Calculate Final Result
Academic Report Card: Vansh Modi
Total Score
85.0 / 500
Percentage
17.00%
Final Grade
F (Fail)
Subject Name
Marks Obtained
Maximum Marks
Mathematics 23.0000 100
Science 19.0000 100
English 30.0000 100
Computer Science 2.0000 100
History 11.0000 100
Final Result: FAIL — Minimum 40% required to pass.


localhost
Student Marks Calculator
Deploy
Student Academic Marks & Grade Calculator
Instructions:
1. Enter the student's full name.
2. Enter marks obtained in each of the 5 subjects (must be between 0 and 100).
3. Click Calculate Final Result to view total marks, percentage, grade, and pass/fail status.
Student Full Name
Vansh Modi
Mathematics (0 - 100)
150.00
Computer Science (0 - 100)
94.00
Science (0 - 100)
82.00
History (0 - 100)
-120.00
English (0 - 100)
79.00
Calculate Final Result
Invalid Input Error: All subject marks must be strictly between 0 and 100.

RESULTS, PREDICTIONS & CONCLUSION

5. Results and Observations:

- **CNN Test Performance:** When evaluated on the 10,000 unseen MNIST test images, the trained CNN achieved a **Final Test Accuracy of ~99.0%** and a **Final Test Loss of ~0.03**.
- **Sample Actual vs. Predicted Labels (Simple Explanation):** The visual comparison below displays 5 correct predictions (top row, green) and 5 incorrect predictions (bottom row, red) from the test set. In simple terms, the CNN classifies clearly written digits with near-perfect reliability. The rare misclassifications occur only when a handwritten digit has missing loops, extreme slant, or ambiguous strokes (for example, an incomplete 9 or 4, or a 5 that closely resembles a 3 or 6), which can even be ambiguous to human eyes.



- **Streamlit App Findings:** The Streamlit web application executes without errors, updates metrics and tables immediately upon button submission, and reliably catches out-of-range scores or empty text inputs.

6. Conclusion:

- **Key Learnings:** Completing this project provided hands-on mastery of the end-to-end deep learning image classification workflow—including tensor normalization, convolutional/pooling feature extraction, training/validation curve diagnostics, and test-set evaluation—alongside building responsive, validated web interfaces in Streamlit.

- **Challenges Faced:**

1. Configuring compatible deep learning (torch/torchvision) and data serialization (pyarrow/numpy) package versions on an Intel (x86_64) macOS environment, along with resolving macOS SSL certificate verification for dataset downloads.
2. Combining both Part A (CNN training pipeline) and Part B (Streamlit web app) into

a single .py script without triggering CNN training on every Streamlit UI interaction, which was solved by separating the workflows into modular functions routed by execution context.

- **Possible Future Improvements:**

1. Adding image data augmentation (random slight rotations, shifts, and zoom) during training to make the CNN even more resilient to heavily tilted handwriting.
2. Extending the CNN architecture with Batch Normalization and deeper blocks to classify complex RGB color objects on the **CIFAR-10** dataset.
3. Integrating an interactive drawing canvas into the Streamlit app so users can draw a digit on screen and receive real-time predictions from the trained CNN model.